

Hoja de Trabajo 2

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [ ]: # %pip install -r requirements.txt
# jupyter nbconvert ws2.ipynb --to html
```

Task 1 - Análisis

Considere que usted está diseñando el sistema de visión para un robot de almacén que debe moverse entre estanterías para recoger productos. El robot tiene dos cámaras frontales:

Inciso 1

Durante una prueba, el robot gira sobre su propio eje para escanear el entorno. El ingeniero junior a tu cargo sugiere usar Homografías para medir la distancia a los objetos mientras el robot gira. ¿Es este un enfoque correcto? Justifique su respuesta utilizando los conceptos de C_1 , C_2 y Paralaje.

Respuesta:

No, porque al ser estanterías se deberá medir distancia mientras el robot solo gira.

- Si el robot gira sobre su eje, el centro de la cámara no se mueve: $C_1 = C_2$ (no hay traslación).
- Cuando $C_1 = C_2$ entonces el paralaje es ≈ 0 .
- Sin paralaje no hay disparidad útil, y por lo tanto se puede triangular profundidad.
- La homografía sirve para relacionar dos vistas bajo rotación o cuando la escena puede aproximarse como plana; es útil para alinear/"stitching", no para medir distancia 3D real en estanterías con objetos a diferentes profundidades.

Con giro puro se puede usar homografías para registrar imágenes, pero no para estimar distancias a objetos en 3D.

Inciso 2

Si el robot comienza a avanzar (traslación) y detectas que la disparidad (d) de una caja aumenta repentinamente entre el frame t y el frame $t+1$, ¿qué puedes inferir sobre la distancia (Z) entre el robot y la caja? ¿Qué riesgo industrial implicaría un error en el cálculo de esta disparidad?

Respuesta:

Por la relación $Z = \frac{fB}{d}$, si $d \uparrow \Rightarrow Z \downarrow$, es decir la caja está más cerca del robot.

Riesgo industrial si d se calcula mal:

- Un error en disparidad produce un error directo en profundidad (Z), causando:
 - Choque con las estanterías/cajas.
 - El robot fallará cuando intente agarrar la caja (está mas cerca/lejos de donde está realmente).
 - Esto a su vez puede llegar a dañar el producto de las estanterías aumento pérdidas monetarias.

Task 2 – Ingeniería de Dimensiones

Como director de proyectos, debe asegurar que los modelos de IA quepan en la memoria de los dispositivos (Edge Computing). Por ello, tiene una imagen de entrada de alta resolución proveniente de una cámara industrial de 1280 x 720 píxeles. Se aplica una capa convolucional con los siguientes hiperparámetros:

- Tamaño del Filtro (F): 5×5
- Padding (P): 2
- Stride (S): 2

Considerando esto, respondan:

Inciso 1

Utilizando la fórmula vista en clase, calcule las dimensiones (Wout,Hout) del Mapa de Características resultante. Muestra el procedimiento.

Respuesta:

Fórmula:

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

Datos:

- $W = 1280$
- $H = 720$
- $F = 5$
- $P = 2$
- $S = 2$

Cálculo en ancho

$$W_{out} = \left\lfloor \frac{1280 - 5 + 2(2)}{2} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{1280 - 5 + 4}{2} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{1279}{2} \right\rfloor + 1$$

$$W_{out} = \lfloor 639.5 \rfloor + 1$$

$$W_{out} = 639 + 1 = 640$$

Cálculo en alto

$$H_{out} = \left\lfloor \frac{720 - 5 + 4}{2} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{719}{2} \right\rfloor + 1$$

$$H_{out} = \lfloor 359.5 \rfloor + 1$$

$$H_{out} = 359 + 1 = 360$$

Resultado

$$(W_{out}, H_{out}) = (640, 360)$$

Inciso 2

¿Qué sucedería con el tamaño de la salida si decides cambiar el Padding a $P=0$ (Valid Padding)?

¿Cómo afectaría esto a la información de los bordes de la imagen (donde suelen estar las referencias de las paredes del almacén)?

Respuesta:

Fórmula:

$$O = \left\lfloor \frac{W - F}{S} \right\rfloor + 1$$

Ancho

$$W_{out} = \left\lfloor \frac{1280 - 5}{2} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{1275}{2} \right\rfloor + 1$$

$$W_{out} = 637 + 1 = 638$$

Alto

$$H_{out} = \left\lfloor \frac{720 - 5}{2} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{715}{2} \right\rfloor + 1$$

$$H_{out} = 357 + 1 = 358$$

Nuevo tamaño

$$(638, 358)$$

Con $P = 0$ (valid padding):

- La convolución hace que la imagen se reduzca naturalmente, porque el kernel no logra meterlos en los bordes.
- El centro del filtro no puede llegar a las esquinas, por lo que esos píxeles se pierden.
- Dado que no debemos hacer de menos la información de los límites, porque también contiene relaciones espaciales.

En el contexto del robot de almacén:

- Las paredes y estanterías podrían aparecer en los extremos del frame.
- Si eliminamos esos píxeles, estamos perdiendo referencias espaciales afectando en el funcionamiento del robot porque no interpreta bien la percepción de límite.

Aunque reduce dimensiones y puede ayudar en memoria, también implica pérdida de información en bordes llegando a perder los detalles periféricos que son importantes para la geometría y orientación del robot.

Task 3 – Criterio de Diseño

En la industria, el balance entre precisión y velocidad es clave. Analice los siguientes escenarios:

Inciso 1

Usted está desarrollando un sistema de detección de grietas microscópicas en motores de avión. ¿Qué combinación de Stride y Pooling recomendaría para no perder detalles críticos en las primeras capas de la red? Justifique técnicamente.

Respuesta:

Para no perder detalles críticos como grietas microscópicas en las primeras capas, se recomendaría:

- **Stride pequeño ($S=1$):** Permite evaluar todos los píxeles sin realizar saltos. Un stride mayor submuestrearía la imagen y omitiría estas anomalías sutiles desde el inicio, por lo que es vital para la alta resolución en fallas minúsculas.
- **Sin Pooling o Mínimo Pooling:** Se debe evitar el Max Pooling en las primeras capas. El Max Pooling reduce la resolución espacial y aporta invarianza a la traslación, descartando detalles estructurales exactos que distinguen una grieta real del ruido normal. Si la RAM fuera un impedimento insalvable, usar ventanas pequeñas (ej. 2×2 con $S=2$).

En conclusión, la primera capa debe enfocarse en una extracción exhaustiva con $Stride = 1$ y omitiendo el *Pooling* inmediato.

Inciso 1

Un cliente le pide que el sistema funcione en un procesador muy limitado (como una cámara inteligente con poca RAM). Explique cómo podrías utilizar el Stride y el Max Pooling estratégicamente para reducir la carga computacional sin eliminar las características más fuertes (activaciones) del Mapa de Características

Respuesta:

Para reducir la carga computacional (RAM y procesamiento) pero mantener las mejores activaciones, combinar ambas técnicas es clave:

- **Max Pooling (ej. ventana 2×2 con $S = 2$):** Esta operación reduce la resolución espacial a la mitad en cada dimensión (dividiendo las operaciones subsecuentes por 4), pero reteniendo la "activación más fuerte" en esa región del mapa de características. Así nos aseguramos de no perder las respuestas más prominentes detectadas por los filtros, como los bordes principales de los objetos de la nave.
- **Stride mayor a 1 (ej. $S = 2$) en convoluciones:** Aplicar stride desde la capa convolucional disminuye inmediatamente el tamaño del Feature Map resultante reduciendo el número de multiplicaciones matriciales drásticamente.

Ambos métodos descartan datos redundantes, reteniendo solo características esenciales. Esto permite procesar imágenes rápidamente con recursos limitados preservando la efectividad.

Task 4 – Implementación Práctica

Con esta parte se busca que puedan comprender la mecánica interna de la operación convolucional sin depender de librerías de alto nivel para la lógica central. Por ello realice lo siguiente:

- Utilizando un lenguaje de programación (Python) y librerías básicas para manejo de matrices (como NumPy), implementa una función llamada `manual_convolution(image, kernel, stride, padding)`.
- **Requisitos de la implementación:**
 - La función debe recibir una matriz 2D (imagen en escala de grises) y un filtro (kernel) de tamaño $N \times N$.
 - Debe aplicar primero el Zero-padding a la imagen de entrada según el valor de P.
 - Debe recorrer la imagen aplicando el producto punto (suma de productos elementales) respetando el Stride indicado.
 - La función debe retornar la matriz resultante (Feature Map).
- **Prueba de validación:** Defina un filtro de detección de bordes verticales (filtro de Sobel o similar) y aplíquelo a una imagen pequeña de prueba.
- **Entrega:** Suba su Jupyter Notebook con comentarios explicando cómo el desplazamiento del kernel afecta el tamaño de la matriz de salida.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

def manual_convolution(image, kernel, stride=1, padding=0):
    """
    Aplica una operación de convolución manual sobre una imagen 2D.
    """
    # Obtener dimensiones
    H, W = image.shape
    F_H, F_W = kernel.shape

    # 1. Aplicar Zero-padding (P)
    if padding > 0:
        image_padded = np.pad(image, pad_width=padding, mode='constant', constant_values=0)
    else:
        image_padded = image

    # 2. Calcular dimensiones de salida esperadas
    # Fórmula:  $O = \text{floor}((W - F + 2P) / S) + 1$ 
    H_out = int(np.floor((H - F_H + 2 * padding) / stride)) + 1
    W_out = int(np.floor((W - F_W + 2 * padding) / stride)) + 1

    # Inicializar el mapa de características
    feature_map = np.zeros((H_out, W_out))

    # 3. Recorrer la imagen con el producto punto respetando el Stride (S)
```

```

# NOTA: El desplazamiento del kernel afecta el tamaño de la matriz de salida.
# Un Stride = 1 avanza el kernel píxel por píxel. Un Stride > 1 causará que
# el kernel "salte" regiones, evaluando menos ventanas y, en consecuencia,
# el tamaño de salida (feature_map) será drásticamente menor.
for i in range(H_out):
    for j in range(W_out):
        h_start = i * stride
        h_end = h_start + F_H
        w_start = j * stride
        w_end = w_start + F_W

        # Extraer la ventana de la imagen original
        window = image_padded[h_start:h_end, w_start:w_end]

        # Aplicar suma de productos (producto punto escalar)
        feature_map[i, j] = np.sum(window * kernel)

    return feature_map

# === Prueba de Validación ===

# 1. Crear una imagen de prueba sencilla (10x10)
# Agregamos una barra vertical blanca en medio para simular un borde
test_image = np.zeros((10, 10))
test_image[:, 4:6] = 255

# 2. Definir kernel de detección de bordes verticales (Filtro de Sobel vertical 3x3)
sobel_v = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]])

# 3. Aplicar convolución manual
# Se prueba con Padding = 1 y Stride = 1 para que la salida conserve su tamaño
output_map = manual_convolution(test_image, kernel=sobel_v, stride=1, padding=1)

# 4. Mostrar imágenes
fig, axs = plt.subplots(1, 3, figsize=(15, 5))
axs[0].imshow(test_image, cmap='gray')
axs[0].set_title('Imagen Original (10x10)')

axs[1].imshow(sobel_v, cmap='gray')
axs[1].set_title('Filtro (Sobel Vertical)')

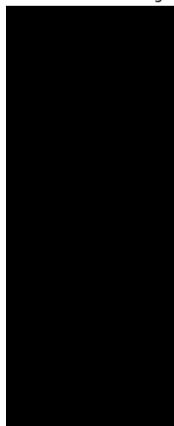
axs[2].imshow(output_map, cmap='gray')
axs[2].set_title('Feature Map de Salida (Bordes Detectados)')

for ax in axs:
    ax.axis('off')

plt.tight_layout()
plt.show()

```

Imagen Original (10x10)



Filtro (Sobel Vertical)



Feature Map de Salida (Bordes Detectados)

